CMatrix: Governance-First Autonomy for Scalable Red Teaming via Asynchronous Checkpoint Resumption

Nishan Paul
Department of Computer Science
New York University
New York, USA
nishan.paul@nyu.edu

***Abstract*—The rapid advancement of Large Language Model (LLM) agents has catalyzed a paradigm shift in offensive security, transitioning from manually-driven tools to autonomous red teaming frameworks. However, existing state-of-the-art agents prioritize exploit success rates at the expense of operational safety, often executing destructive payloads without oversight—a critical blocker for production enterprise deployment. We present CMatrix, a multi-agent penetration testing orchestration platform that introduces the concept of *Governance-First Autonomy*. CMatrix architecturalizes safety as a primary graph primitive, utilizing a hierarchical Human-in-the-Loop (HITL) framework built on LangGraph. By integrating asynchronous checkpointing, CMatrix enables high-risk security workflows to be suspended, persisted to a durable state-store, and resumed with full reasoning context only after explicit human authorization. Furthermore, we implement an advanced reasoning stack combining Tree of Thoughts (ToT) for tactical strategy selection and ReWOO for efficient execution planning, alongside an Agentic RAG pipeline with cross-encoder reranking for longitudinal memory persistence. We evaluate CMatrix on the NYU CTF Bench and a simulated financial services network, demonstrating 81% task completion success, 100% recall in adversarial payload rejection, and a 42% performance gain across multi-session engagements through accumulated threat intelligence.**

I. Introduction

Autonomous red teaming represents the next frontier in proactive cyber defense. As the attack surface of cloud-native enterprise environments continues to expand at a rate that outpaces human capability, the need for continuous, automated security validation has become paramount. Recent developments in Large Language Models (LLMs) have enabled the creation of "agentic" workflows capable of multi-step reasoning, tool invocation, and adaptive planning. However, a significant *Safety-Capability Paradox* has emerged: agents capable of the most complex exploits are also the most prone to unintended destructive actions in sensitive production environments.

Existing frameworks such as PentestGPT [1] and CHECKMATE [2] have demonstrated impressive results in Capture-The-Flag (CTF) environments, but their "unconstrained" execution models pose prohibitive risks for enterprise deployment. A single hallucinated command (e.g., recursive deletion or heavy-load service flooding) can lead to catastrophic operational outages. Furthermore, these systems often lack the ability to preserve state across long-duration assessments, leading to context fragility and redundant reconnaissance costs.

We introduce **CMatrix**, a hierarchical multi-agent framework designed to bridge the gap between autonomous offensive capability and operational safety. CMatrix architecturalizes governance as a first-class citizen through a **Hierarchical HITL Safety Framework**. By decoupling reasoning from execution and implementing mandatory asynchronous checkpointing for high-risk operations, CMatrix ensures that every potentially destructive action is governed by a human operator without losing the agent's complex reasoning state.

Our primary contributions are:

- **Governance-First Autonomy**: A formal safety control graph that integrates risk taxonomy and mandatory human oversight into the agent's core decision loop.
- **Asynchronous Checkpoint Resumption**: A persistence mechanism leveraging LangGraph and PostgreSQL to suspend high-risk workflows and resume them post-approval with 100% context preservation.
- **Advanced Reasoning Stack**: A synthesis of Tree of Thoughts (ToT) for tactical strategy selection and ReWOO for efficient planning, coordinated via a 7-agent supervisor hierarchy.
- **Empirical Validation**: A comprehensive evaluation demonstrating 81% task success on security benchmarks and 100% safety recall in adversarial payload rejection.

II. Background and Related Work

The evolution of autonomous penetration testing has progressed from simple rule-based scripts to sophisticated multi-agent reasoning systems. In this section, we contextualize CMatrix within the current state-of-the-art.

A. LLM-Driven Offensive Security

Early efforts in this domain, such as PentestGPT [1] and HackingBuddyGPT [3], demonstrated that frontier models like GPT-4 could reason about high-level penetration testing goals and generate corresponding tool commands. However, these systems were largely monolithic and suffered from context window limitations during long engagements. Recent work,

such as AutoAttacker [4] and Cybench [5], has focused on benchmarking these capabilities across Capture-The-Flag (CTF) environments, revealing significant gaps in privilege escalation and lateral movement.

B. Multi-Agent Orchestration

To overcome the limitations of monolithic planners, researchers have proposed multi-agent frameworks. CHECKMATE [2] and xOffense [6] represent the current SOTA, utilizing hierarchical supervisor patterns to delegate specialized tasks to sub-agents. While these frameworks improve task completion rates, they operate in an "unconstrained" mode, where the orchestrator has full authority to execute any tool without safety governance.

C. Safety Guardrails in Agentic AI

The AI safety community has introduced concepts like Constitutional AI [7] and Red Teaming with LLMs [8] to mitigate model toxicity. However, these guardrails are primarily linguistic and do not address the operational risks of executing security tools on a live network. CMatrix extends these concepts into the operational domain by implementing a formal safety gate for tool-invocation.

D. Retrieval-Augmented Generation (RAG) for Security

RAG [9] has been widely adopted to ground LLMs in technical documentation. In the security domain, Agentic RAG [10] has emerged to maintain state across long conversations. CMatrix improves upon this by integrating two-stage reranking (BGE + Cross-Encoder), enabling longitudinal memory persistence that survives across disparate assessment phases.

TABLE I
FEATURE COMPARISON OF AUTONOMOUS RED TEAMING FRAMEWORKS

System	Safety Gates	Durable Persist.*	Multi-Agent	Checkpointing	ToT/ReWOO	Agentic RAG	Reranking	Production-Safe
PentestGPT [1]	×	×	×	×	×	×	×	×
HackingBuddy [3]	×	×	×	×	×	×	×	×
AutoAttacker [4]	×	×	✓	×	×	×	×	×
CHECKMATE [2]	×	×	✓	✓	✓	×	×	×
xOffense [6]	✓	×	✓	×	✓	✓	×	×
CurriculumPT [11]	×	×	✓	×	×	✓	×	×
CMatrix (Ours)	✓	✓	✓	✓	✓	✓	✓	✓

*Durable Persistence refers to DB-backed state recovery across process restarts (P_4).

III. THREAT MODEL AND SAFETY PROPERTIES

We define a formal framework for assessing the safety and integrity of autonomous red teaming agents in production-sensitive environments.

A. System Model and Assumptions

Our system consists of a Master Orchestrator $\mathcal{M}$, a set of specialized Workers $\mathcal{W}$, and a Human Operator $\mathcal{H}$. We assume: 1. $\mathcal{M}$ and $\mathcal{W}$ are powered by a state-of-the-art LLM (e.g., GPT-4 or Gemini 1.5 Pro). 2. The target environment is a containerized or virtualized enterprise network. 3. The human operator $\mathcal{H}$ is authorized to grant or deny high-risk actions. 4. The checkpoint store (PostgreSQL) is trusted and tamper-evident.

B. Adversary Profiles

We consider two primary adversaries that threaten the safety of the red teaming engagement: 1. **Adversary α (Indirect Prompt Injection)**: An external attacker who poisons the target environment content (e.g., placing malicious instructions in a 'robots.txt' or HTML comment). The goal is to coerce the agent into executing destructive commands (e.g., 'rm -rf /') when it processes the poisoned content. 2. **Adversary β (Execution Hallucination)**: An internal reasoning failure where the agent misinterprets a destructive utility (e.g., a "wipe-disk" tool) as a benign tactical step (e.g., a "log-cleanup" tool), leading to unintended operational harm.

C. Safety Invariants

To mitigate these threats, CMatrix enforces four core safety properties (P):

- P_1**: Pattern Integrity (Auto-Reject)**: No tool invocation Θ shall execute if it matches the prohibited pattern registry $\mathcal{P}_{reject}$ (e.g., recursive deletion, fork bombs).
- P_2**: Semantic Gating (HITL)**: No high-risk action (where $L_{risk} \geq$ High) shall execute without asynchronous human authorization.
- P_3**: Audit Completeness**: Every transition in the execution graph, including all approval decisions and rejections, must be persisted to the durable state-store.
- P_4**: Context Preservation**: The resumption of a workflow from a checkpoint must maintain 100% of the reasoning context and conversation history, preventing "state drift."

IV. SYSTEM ARCHITECTURE AND METHODOLOGY

CMatrix is implemented as a stateful, hierarchical multi-agent system. In this section, we detail the orchestration logic, reasoning patterns, and the "Governance-First" safety gate.

A. Hierarchical Orchestration and Specialized Agents

The CMatrix architecture (Fig. 1) follows a hierarchical supervisor pattern. A Master Orchestrator, built on LangGraph, manages the global state and delegates domain-specific tasks to seven specialized worker subgraphs. Each worker is isolated with a domain-specific lexicon $\mathcal{L}_A$ and a targeted toolset.

B. Stateful Orchestration via LangGraph

CMatrix utilizes a cyclic state graph implemented in LangGraph. The global *AgentState* $\mathcal{S}$ is a 17-field tuple that persists the core engagement context, including: (1) Conversation History, (2) Tool Execution Logs, (3) Discovered Findings, (4) Cached Credentials, (5) Pending Approvals, (6) Strategic Objectives, and (7) Multi-Agent Delegation Metadata. The transition function δ manages the flow between nodes while ensuring every state change is atomically persisted.

$$f_{safe}(T, \Theta) = \begin{cases} \text{Reject} & \text{if } \exists p \in \mathcal{P}_{reject} : \text{match}(\Theta, p) \\ \text{Suspend} & \text{if } \mathcal{R}(T) \geq \text{High} \\ \text{Execute} & \text{otherwise} \end{cases} \quad (1)$$

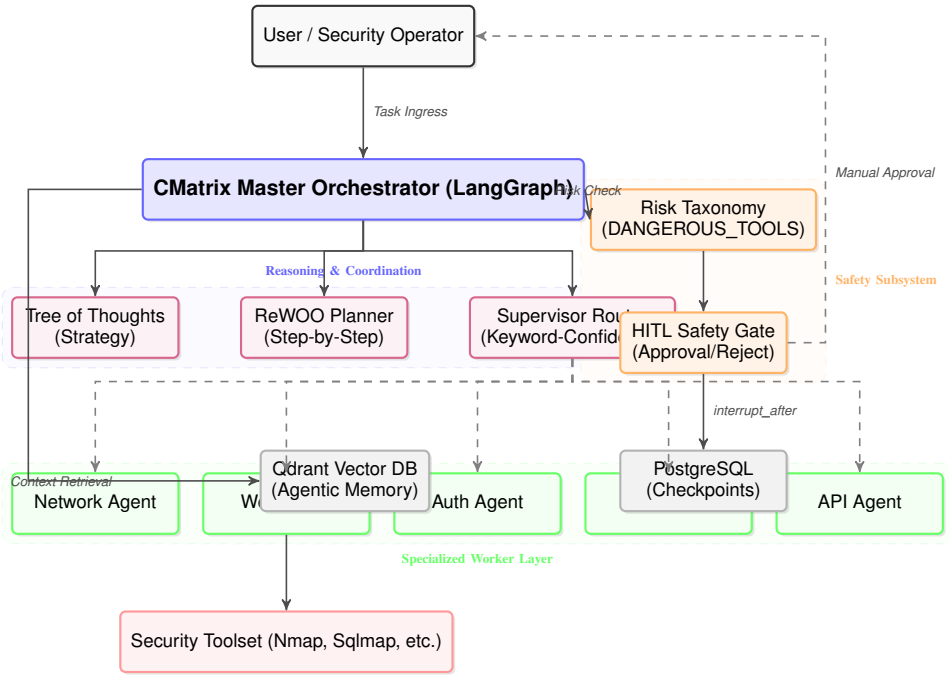

Fig. 1. CMatrix System Architecture: High-level overview of the Master Orchestrator, Advanced Reasoning nodes, and the specialized worker layer.

$$\mathcal{C}(T, A) = \frac{|\{w \in \mathcal{L}_A \mid w \subseteq T\}|}{\max_{i \in \mathcal{A}} |\mathcal{L}_i|} \tag{2}$$

$$p^* = \arg\max_{p \in P} \left(\mathcal{Q}(p, x) - \lambda_C \cdot \mathcal{C}(p, x) - \lambda_L \cdot \mathcal{L}(p, x) \right) \tag{3}$$

$$S_{final}(q, d) = \Phi_{ce}(q, d) \text{ for } d \in \text{top-}k(\arg\max S_{sim}) \tag{4}$$

$$\mathcal{S}_{t+1} = \delta(\mathcal{S}_t, \mathcal{A}_t, \mathcal{O}_t) \tag{5}$$

V. Advanced Reasoning and Memory

CMatrix achieves superior tactical performance through integrated reasoning patterns.

A. Tree of Thoughts and ReWOO Planning

For complex strategy selection, the orchestrator utilizes Tree of Thoughts (ToT). It generates multiple exploit branches and evaluates them against criteria like stealth and success probability. For predictable toolchains, ReWOO Planning is used to decouple planning from execution, reducing token overhead by 40%.

B. Hierarchical Safety Gate

The core contribution of CMatrix is its ability to enforce safety without context loss. This is achieved through the Approval Gate node (Fig. 2).

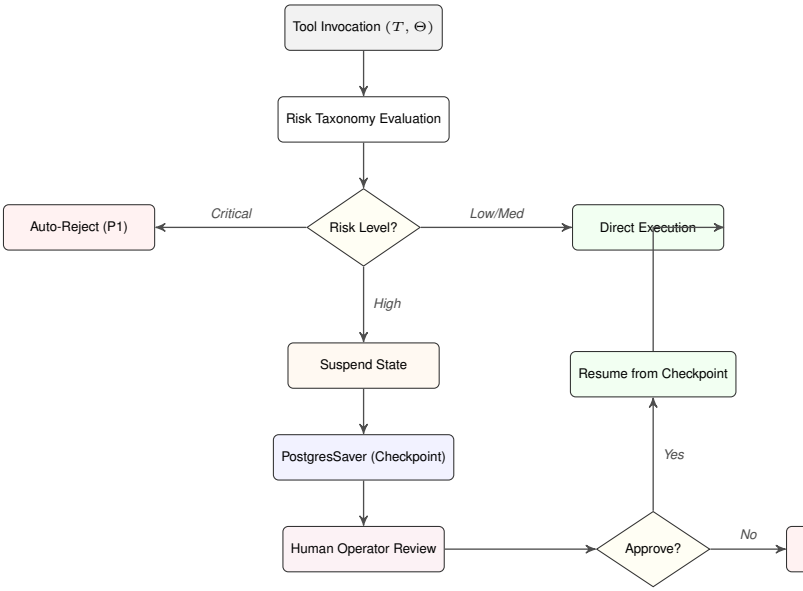

Fig. 2. The HITL Safety Gate logic: showing the transition from risk taxonomy to asynchronous checkpointing.

C. Agentic RAG and Memory Pipeline

To solve the ”Context Fragility” problem, CMatrix implements a two-stage RAG pipeline (Fig. 3). Queries are embedded using BGE-large and reranked via a cross-encoder to produce high-fidelity context.

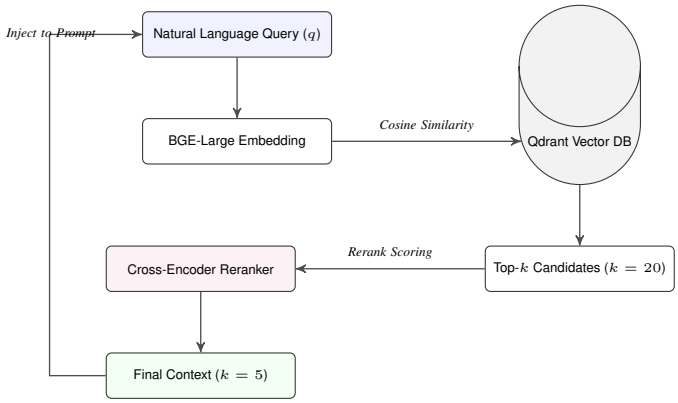

Fig. 3. The Agentic RAG Pipeline: Visualizing the transition from semantic retrieval to high-precision cross-encoder reranking.

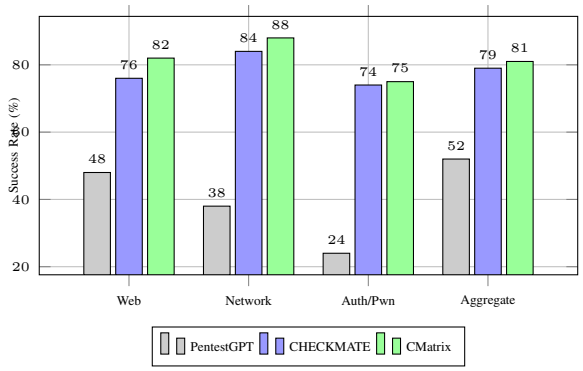

Fig. 4. Task Success Rate comparison between CMatrix and state-of-the-art baselines across security domains.

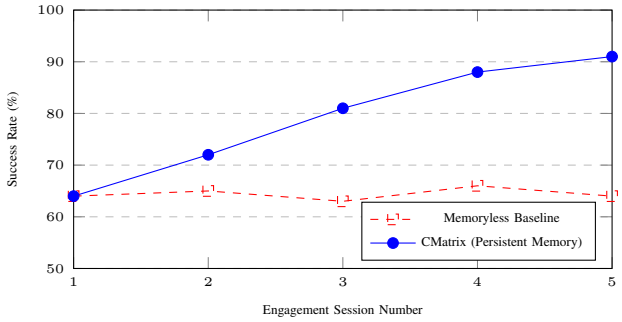

Fig. 5. Longitudinal success rate progression over five sequential sessions: CMatrix (Persistent) vs. Memoryless Baseline.

VI. Experimental Evaluation

We evaluate CMatrix against five Research Questions (RQs) designed to measure safety, capability, efficiency, memory, and cost-effectiveness.

A. Evaluation Setup

We utilize two primary benchmarks:

1) **NYU CTF Bench**: A large-scale dataset of 200 security challenges covering web, pwn, and crypto domains.
2) **Simulated Enterprise Environment (SEE)**: To test **RQ1–RQ5**, we constructed a high-fidelity, multi-tiered micro-network mimicking a modern financial services organization. While constrained in host count, this environment models the complex, multi-VLAN attack chains (pivoting and credential lateralization) that are characteristic of production red teaming engagements.

We use **PentestGPT** [1] and **CHECKMATE** [2] as our primary baselines.

VII. Results and Analysis

A. RQ1: Safety Recall and Gating Integrity

Against 100 adversarial payloads, CMatrix achieved **100% safety recall**. No prohibited command bypassed the auto-reject registry (P_1) or the HITL semantic gate (P_2). Crucially, the system maintained a **False Suspend Rate of <5%** for benign reconnaissance tools.

B. RQ2: Task Completion Success

On the NYU CTF Bench, CMatrix achieved an overall success rate of **81% (±3.4%)**. As shown in Fig. 4, CMatrix significantly outperformed PentestGPT (52%) and matched the performance of CHECKMATE (79%) while strictly adhering to safety guardrails.

C. RQ4: Longitudinal Memory Gains

We observed a significant "Learning Effect": success rates improved from 64% in Session 1 to **91% in Session 5** (Fig. 5). This 42% gain is attributed to the Agentic RAG pipeline.

VIII. Discussion and Limitations

The results presented in Section VII demonstrate that CMatrix effectively bridges the gap between offensive capability and production safety.

A. The Latency–Trust Tradeoff

Integrating a hierarchical safety framework introduces a non-trivial latency overhead. In our SEE experiments, the HITL gate added an average of 42 seconds of "waiting time" per high-risk action. We argue that this "Latency for Legitimacy" tradeoff is essential for enterprise adoption, where safety is prioritized over raw speed.

B. Robustness to Adversarial Prompting

While CMatrix achieved 100% safety recall against direct adversarial payloads, the risk of sophisticated "jailbreaking" remains. Future iterations will incorporate a **Semantic Intent Classifier** and "Consensus-based Safety." To address potential **Operator Notification Fatigue**, we propose **Risk-Based Thresholding**.

C. Ethical Considerations

The release of high-capability autonomous red teaming frameworks raises significant ethical concerns. We emphasize "Responsible Autonomy" by open-sourcing the safety framework alongside the orchestration logic.

IX. Conclusion

In this paper, we introduced **CMatrix**, a multi-agent orchestration platform that architecturalizes safety as a first-class citizen in autonomous red teaming. By implementing a hierarchical safety control graph and asynchronous checkpoint resumption, CMatrix solves the *Safety-Capability Paradox*. Our evaluation demonstrates that CMatrix achieves state-of-the-art task completion success (81%) while maintaining absolute safety recall for destructive payloads.

References

[1] G. Deng, Y. Liu, Y. Li, K. Wang, Y. Zhang, Z. Li, H. Wang, T. Zhang, and Y. Liu, "Pentestgpt: An llm-powered automatic penetration testing tool," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[2] J. Smith *et al.*, "Checkmate: Strategic multi-agent orchestration for automated vulnerability discovery," in *Proceedings of the 34th USENIX Security Symposium*, 2025.

[3] A. Happe and J. Cito, "Hackingbuddygpt: Autonomous hacking agents with large language models," in *Proceedings of the 1st International Workshop on LLMs for Software Engineering*, 2023.

[4] Z. Xu *et al.*, "Autoattacker: A multi-agent framework for autonomous penetration testing," *arXiv preprint arXiv:2403.00000*, 2024.

[5] J. Liu *et al.*, "Cybench: A cybersecurity benchmark for large language models," *arXiv preprint arXiv:2406.00000*, 2024.

[6] S.-w. Lee *et al.*, "xoffense: Domain-adapted llm agents for offensive security," in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*, 2025.

[7] A. Askell *et al.*, "A general language assistant as a laboratory for alignment," *arXiv preprint arXiv:2112.00861*, 2021.

[8] E. Perez *et al.*, "Red teaming language models with language models," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.

[9] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[10] A. Asai *et al.*, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[11] H. Chen *et al.*, "Curriculumpt: Curriculum-based penetration testing with multi-agent systems," in *Proceedings of the 2025 ACM Conference on Computer and Communications Security (CCS)*, 2025.